

Graph reduction using combinators

Mário Florido

March 17, 2025

Bibliography

- *Foundations for Functional Programming*, Section 7, Lawrence C. Paulson.
- *Implementation of Functional Languages*, Chapters 10, 11, 12, 16
- Video presentation @ Strange Loop Conf:
<https://youtu.be/GawiQQCn3bk>

Strict vs. non-strict semantics

How should we evaluate $((\lambda x. M) N)$?

Strict semantics

Call-by-value evaluate N once (**even if not used**)

Non-strict semantics

Call-by-name evaluate N zero or multiple times (**every time it is used**)

Call-by-need, lazy evaluation evaluate N zero or once (**only if it is used**)

Strict vs. non-strict semantics (cont.)

Strict semantics

- Usual in all imperative languages
- Also Scheme, Standard ML, OCaml and F#
- But: non-strict semantics is used in particular cases
 - logic operations `&&`, `||`
 - special parameters declared *call-by-name*
 - some data structures (e.g. *streams*)
 - *macros*

Non-strict semantics

- *Call-by-name* is too inefficient for general use
- *Call-by-need/lazy evaluation* some pure functional languages (Miranda, Clean, Haskell)

Difficulties in implementation

- *Call-by-name* is inefficient: duplicates computations
- *Call-by-need/lazy evaluation*:
 - we need to represent suspended computations (*thunks*)
 - after evaluation *update* the thunk with its result
 - share the result of the thunk for every use

Examples

```
sqr =  $\lambda x.$  mult x x  
main = sqr (sqr 5)
```

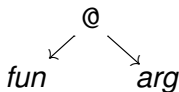
Let us compute the expression “main”.

Call-by-name reduction

sqr (sqr 5) $\rightarrow$ mult $\underbrace{(\text{sqr } 5) (\text{sqr } 5)}_{\text{duplication}}$
 $\rightarrow$ mult (sqr 5) (sqr 5)
 $\rightarrow$ mult (mult 5 5) (sqr 5)
 $\rightarrow$ mult 25 (sqr 5)
 $\rightarrow$ mult 25 (mult 5 5)
 $\rightarrow$ mult 25 25
 $\rightarrow$ 625

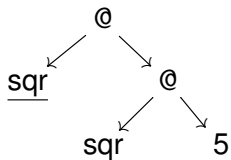
Graph reduction

- Represent the expression as a graph where applications are binary nodes:

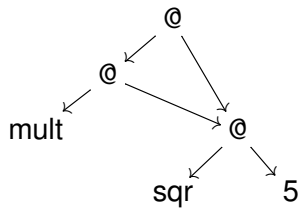


- Look for the **outermost** and **leftmost** *redex*
- After reducing a sub-term: **update the graph** with the result

Example

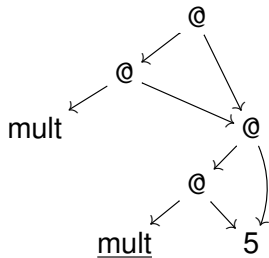


$\Rightarrow$

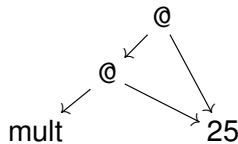


$\Rightarrow$

$\Rightarrow$



$\Rightarrow$



$\Rightarrow$

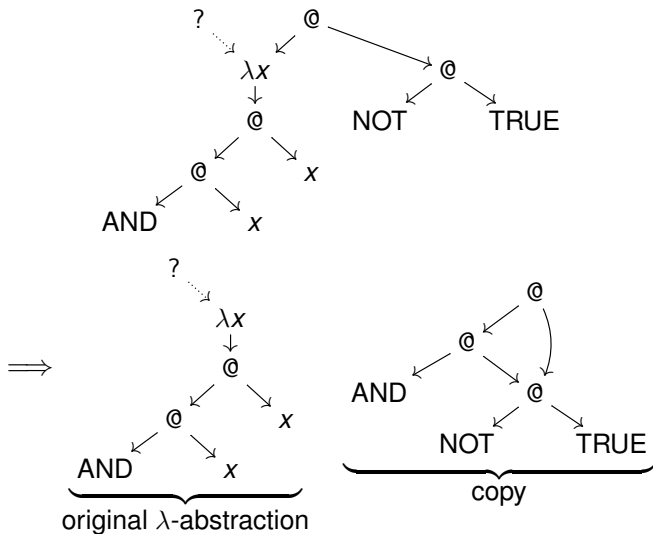
$\Rightarrow$

625

Observations

- Sharing of sub-graphs avoid duplication of computations
- The graph can be implemented in memory as a linked data structure
- Each re-write step modifies pointers in this structure
- Problem: we need to copy the body of the function when we perform a β -reduction

Example



Avoiding copying

We can avoid the need for copying if we use **combinators** instead of λ -terms.

Combinators

The terms of *combinatory logic* are

- 1 variables $x, y, z \dots$;
- 2 constants **S**, **K**, **I**;
- 3 applications $(M\ N)$ where M e N are terms.

Examples:

$$((\mathbf{S}\ x)\ \mathbf{I}) \quad ((\mathbf{S}\ \mathbf{K})\ \mathbf{K}) \quad ((\mathbf{S}\ (\mathbf{K}\ \mathbf{S}))\ \mathbf{K})$$

As in the the λ -calculus, we omit parenthesis by following the convention that applications associate to the left, e.g.

$$\mathbf{S}\ \mathbf{K}\ \mathbf{K} \equiv ((\mathbf{S}\ \mathbf{K})\ \mathbf{K})$$

Combinators (cont.)

Reduction rules $\rightarrow_w$ (*weak reduction*):

$$\mathbf{I} P \rightarrow_w P$$

$$\mathbf{K} P Q \rightarrow_w P$$

$$\mathbf{S} P Q R \rightarrow_w P R (Q R)$$

Example:

$$\mathbf{S} \mathbf{K} \mathbf{K} x \rightarrow_w \mathbf{K} x (\mathbf{K} x) \rightarrow_w x$$

Weak head normal form

A combinator term of the form

$$H E_1 E_2 \dots E_n$$

is in **weak head normal form** if

- 1 H is a constant combinator or a variable; and
- 2 $H E_1 E_2 \dots E_k$ is not a *redex* for any $k \leq n$

In WHNF:

$$\begin{array}{c} x \\ \mathbf{I} \\ \mathbf{S} (\mathbf{K} x) \end{array}$$

Not in WHNF:

$$\begin{array}{c} \mathbf{K} x \\ \mathbf{K} \mathbf{I} y \\ \mathbf{S} (\mathbf{K} x) (\mathbf{I} \mathbf{I}) y \end{array}$$

Translation into combinators

We can translate any λ -term into combinators using two transformations:

- $(-)_{CL}$ converts a λ -term into a combinator term;
- λ^*x auxiliary transformation to abstract a variable.

$$(x)_{CL} \equiv x$$

$$(MN)_{CL} \equiv (M)_{CL}(N)_{CL}$$

$$(\lambda x. M)_{CL} \equiv \lambda^*x.(M)_{CL}$$

$$\lambda^*x. x \equiv \mathbf{I}$$

$$\lambda^*x. P \equiv \mathbf{K} P \qquad x \notin fv(P)$$

$$\lambda^*x. P Q \equiv \mathbf{S}(\lambda^*x. P)(\lambda^*x. Q)$$

Example translation

$$\begin{aligned}(\lambda xy. y\ x)_{CL} &\equiv \lambda^*x. \lambda^*y. (y\ x)_{CL} \\ &\equiv \lambda^*x. \lambda^*y. (y\ x) \\ &\equiv \lambda^*x. \mathbf{S}(\lambda^*y. y)(\lambda^*y. x) \\ &\equiv \lambda^*x. (\mathbf{SI})(\mathbf{K}\ x) \\ &\equiv \mathbf{S}(\lambda^*x. \mathbf{SI})(\lambda^*x. \mathbf{K}\ x) \\ &\equiv \mathbf{S}(\mathbf{K}(\mathbf{SI}))(\mathbf{S}(\lambda^*x. \mathbf{K})(\lambda^*x. x)) \\ &\equiv \mathbf{S}(\mathbf{K}(\mathbf{SI}))(\mathbf{S}(\mathbf{K}\mathbf{K})\mathbf{I})\end{aligned}$$

Properties of the translation

- $(M)_{CL}$ is equivalent to M (see the bibliography — *Foundations of FP*)
- The translation does *not* preserve normal forms (not important for an implementation)
- The translation λ^* may duplicate the number of applications
- The combinator term can be *exponentially larger* than the original λ -term
- The translation can be improved to *quadratic complexity* using more combinators (Turner, 1979)
- We can also need to add combinators for integers, arithmetic operations and the *fixed point combinator* (for recursion)

Improved translation

Extra combinators (Turner):

$$\mathbf{B} P Q R \rightarrow_w P (Q R)$$

$$\mathbf{C} P Q R \rightarrow_w (P R) Q$$

Improved translation (cont.)

$$(x)_{CL} \equiv x$$

$$(MN)_{CL} \equiv (M)_{CL}(N)_{CL}$$

$$(\lambda x. M)_{CL} \equiv \lambda^T x. (M)_{CL}$$

$$\lambda^T x. x \equiv \mathbf{I}$$

$$\lambda^T x. P \equiv \mathbf{K} P \qquad x \notin fv(P)$$

$$\lambda^T x. P x \equiv P \qquad x \notin fv(P)$$

$$\lambda^T x. P Q \equiv \mathbf{B} P (\lambda^T x. Q) \qquad x \notin fv(P)$$

$$\lambda^T x. P Q \equiv \mathbf{C} (\lambda^T x. P) Q \qquad x \notin fv(Q)$$

$$\lambda^T x. P Q \equiv \mathbf{S} (\lambda^T x. P) (\lambda^T x. Q) \qquad x \in fv(P), x \in fv(Q)$$

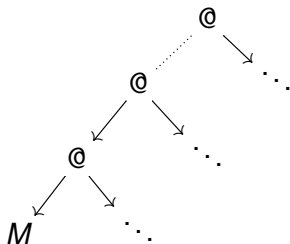
Example revisited

$$\begin{aligned}(\lambda xy. y\ x)_{CL} &\equiv \lambda^T x. \lambda^T y. (y\ x)_{CL} \\ &\equiv \lambda^T x. \lambda^T y. (y\ x) \\ &\equiv \lambda^T x. \mathbf{C}(\lambda^T y. y)\ x \\ &\equiv \lambda^T x. \mathbf{CI}\ x \\ &\equiv \mathbf{CI}\end{aligned}$$

Exercise: Verify that **CI** behaves like the lambda term $\lambda xy. y\ x$ when applied to two arguments.

Graph reduction with combinators

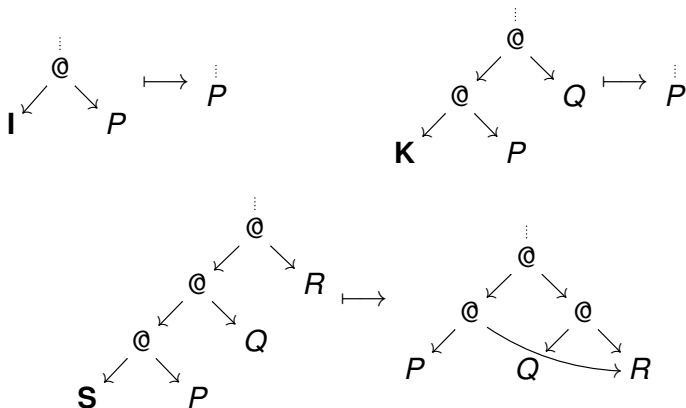
Step 1 (**Unwind**): Find the leftmost, outermost *redex* by descending left on the spine of @-nodes:



such that M is not a @-node

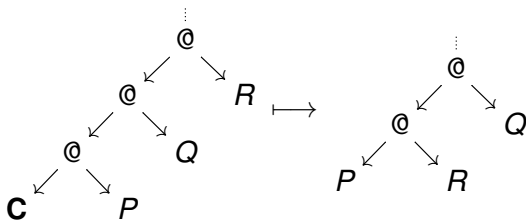
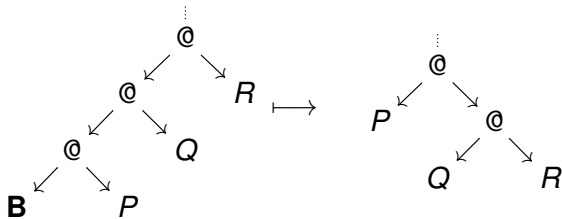
Graph reduction with combinators (cont.)

Step 2 (**Rewrite**): If we find a *redex*, perform local re-writing.



Note the *sharing* of **R** in the reduction for **S**.

Graph reduction with combinators (cont.)



Graph reduction with combinators (cont.)

If there are not enough arguments for the redex in Step 2, then the graph is in *weak head normal form*.

Graph reduction with combinators (cont.)

Step 3: Repeat step 1 and continue until we reach WHNF.

Graph reduction with combinators (cont.)

Strict primitive operations (e.g. ifzero, +, *, etc.) must recursively evaluate their arguments before reducing.

Reduction example

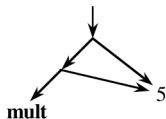
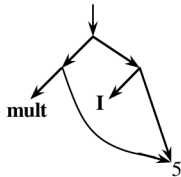
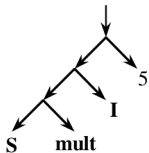
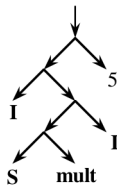
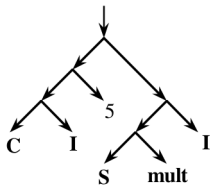
Let us translate

let *sqr* = $\lambda x. x * x$ **in** *sqr* 5

into combinators:

$$\begin{aligned} & (\lambda^T f. f\ 5) (\lambda^T x. \mathbf{mul}\ x\ x) \\ \equiv & \mathbf{C15}\ (\mathbf{S}\ (\lambda^T x. \mathbf{mul}\ x) (\lambda^T x. x)) \\ \equiv & \mathbf{C15}\ (\mathbf{S}\ \mathbf{mul}\ \mathbf{I}) \end{aligned}$$

Reduction example (cont.)



Historical context

- The translation into combinators was used in implementation of *Miranda* in the early 1980s
- Early implementations of Haskell in 1990s used translation into *supercombinators* (GHC, Gofer)
- Special *graph reductions machines* were also built between 1980-1990 (SKIM, NORMA, GRIP)
- No longer state-of-art:
 - special purpose hardware cannot compete with hardware advances in general-purpose CPUs
 - better compilation techniques employ program transformations of the functional program and run on off-the-shelf general purpose architectures
- But is *still* used e.g. in MicroHs:
<https://github.com/augustss/MicroHs>